

Real-Time Depth-of-Field Rendering Using Point Splatting on Per-Pixel Layers

Sungkil Lee¹, Gerard Jounghyun Kim², and Seungmoon Choi¹

¹Haptics and Virtual Reality Laboratory, Computer Science and Engineering, POSTECH, Korea

²Digital eXPerience Laboratory, Computer Science and Engineering, Korea University, Korea

Abstract

We present a real-time method for rendering a depth-of-field effect based on the per-pixel layered splatting where source pixels are scattered on one of the three layers of a destination pixel. In addition, the missing information behind foreground objects is filled with an additional image of the areas occluded by nearer objects. The method creates high-quality depth-of-field results even in the presence of partial occlusion, without major artifacts often present in the previous real-time methods. The method can also be applied to simulating defocused highlights. The entire framework is accelerated by GPU, enabling real-time post-processing for both off-line and interactive applications.

Categories and Subject Descriptors (according to ACM CCS): I.3.3 [Computer Graphics]: Picture/Image Generation

1. Introduction

The finite aperture of a real lens system gives rise to a circular region of blur in the image, called the *circle of confusion* (CoC) [PC81]. Focused objects appear to be sharp within the distance range, called the *depth-of-field* (DOF), around the focal plane, whereas the rest looks blurred with respect to distances to the focal plane. The DOF effect is generally known to play an important role in photorealism, depth perception, and attention attraction.

A number of methods for rendering DOF effects with the pinhole camera model have been developed in the past decades. Among them, the multi-pass rendering methods [CPC84, HA90] simulate most accurate DOF effects. However, their heavy rendering cost has led to the appearance of faster methods that use post-filtering in exchange for quality.

One of severe quality losses common in the post-filtering methods is the lack of the *partial occlusion* effect (the boundaries of a foreground object smoothly spread and seem semitransparent over a focused area). This problem is primarily because information behind foreground objects is not rendered in a typical pinhole image. Although a few methods mitigated the artifact by extrapolating the exterior boundaries of hidden areas [BTCH05, KLO06, KS07], their quality still falls short of the accurate partial occlusion effect achievable with the exact information on the occluded area.



Figure 1: Two example images generated by our DOF rendering method. The upper image shows no intensity leakage, and the lower image shows a partial occlusion effect with defocused highlights of a hexagonal aperture.

This paper presents a real-time high-quality DOF rendering algorithm that provides the convincing partial occlusion effect without other major artifacts such as the intensity leakage. Two representative examples rendered by our method

are shown in Figure 1. To obtain the missing information on the occluded area, an additional pinhole image is rendered and combined with the visible image in a way to create proper partial occlusion effects. The composition with the hidden image requires a layered representation such as the scatter method [PC81] (also called the splatting) or the composition of discrete layers [BBGK02, KS07]. However, the two methods have some difficulties to be directly used for our purpose. Since the scatter method requires costly *depth sorting* of entire pixels, its real-time use is still challenging. The layer methods well approximate the depth sorting using a few discrete layers, but have another quality problem called the *discretization* artifact [BTCH05]. This artifact refers to a band-like artifact appearing in objects separated across multiple layers that do not reflect local depth variation.

Our algorithm is a GPU-based method that combines the advantages of the scatter and layered approaches. Instead of globally defined layers, our method defines three per-pixel layers according to the depth order of a source pixel (scattered from a pinhole image) relative to the destination pixel (to be blurred). This allows even a long object to be decomposed into coherent layers. The visible and hidden images are separately splatted to the per-pixel layers, and then properly blended. As a consequence, our algorithm can produce a high-quality partial occlusion effect. In particular, a sharply focused area occluded by the foreground is *semitransparently* represented with its *accurate colors*.

Another feature of our framework is the amenability to the simulation of photorealistic *bokeh* (distortion of a defocused area) and *defocused highlights* (specular highlights standing out more from blurred areas). These effects can be easily created using the texturing extension of GPU point sprites.

Major contributions of this paper can be summarized as: (1) a new DOF rendering algorithm using the per-pixel layer decomposition, (2) a method to generate the partial occlusion effect using an occluded pinhole image, (3) a real-time GPU implementation of the intensity scatter algorithm, and (4) its application to the bokeh and defocused highlights.

2. Related Work

Multi-Pass DOF Rendering The multi-pass rendering approach such as the distributed ray tracing [CPC84] and the accumulation buffer method [HA90] uses stochastic sampling of different rays within a lens of a finite aperture and results in most accurate DOF effects including the partial occlusion. Since their multi-pass structure requires heavy rendering cost proportional to model complexity, the multi-pass approach is generally inappropriate to real-time rendering. A notable recent advance in this approach is a technique called the “time-continuous triangles” [AMMH07] that is extended from the accumulation buffer method using a reduced number of samples for improved rendering speed. However, the work is still conceptual and requires considerable modifications of the existing graphics hardware to be implemented.

DOF Rendering Using Post-Filtering Another major approach uses post-filtering based on the *gather* [Rok96, MvL00, Dem03, RTI03, BFSC04, Sch04, KLO06, ZCP07, EH07], mostly aiming at real-time performance by utilizing fast texture look-up capability in the GPU. In this approach, the blurring behavior (i.e., the overlapping CoCs) of a lens is approximated by spatially filtering a pinhole image according to the CoC sizes computed from depths. Most of them exhibit major artifacts such as intensity leakage (the intensities of sharp pixels flown onto a background) and the lack of the partial occlusion effect. A few recent methods reduced the intensity leakage problem [BFSC04, KLO06, ZCP07], but to our knowledge, the unconvincing partial occlusion effect still remains to be a problem.

An alternative post-filtering approach is based on the *scatter* [PC81], which can correctly simulate the blurring behavior of overlapping CoCs. Here, the intensity of a source pixel in a pinhole image is distributed to its CoC sprite. The sprites are blended from farther to nearer, and then the image is normalized such that the alpha value for each pixel becomes one. Since sharp pixels are not scattered to the neighborhood, intensity leakage is absent inherently. However, this method also involves the expensive depth sorting (even with a recent GPU), and suffers from the lack of partial occlusion. Our technique was inspired by the scatter method, but has been improved in terms of rendering speed and quality.

DOF Rendering Based on Layer Composition As an approximate depth sorting technique, a few methods where several separately blurred layers are composited have been proposed. An early method [Sco94] used object-space grouping, but cannot be applied to long objects that do not allow object-level partitioning. This limitation was addressed in two later (post-processing) methods [BBGK02, KS07] where several sub-images are extracted from a pinhole image and treated as layers. While these two methods are also free from the intensity leakage, additional two problems are also present with them.

The first problem arises since the two methods allow layer partitioning within an object that can cause the discretization artifact. Despite the special image processing [BTCH05] (the later remedy for [BBGK02]) or the matting technique [KS07], the image quality can be degraded when identical pixels are repeatedly included in multiple layers. In comparison, our method adaptively decomposes layers using local depth variation, and the discretization artifact rarely occurs unless different objects are located too close to each other. The second limitation is that the area filled by the two methods behind foreground layers represents the blurred color of boundaries rather than the accurate (possibly sharp) colors of the occluded area. This artifact becomes more apparent with a large CoC that cannot be covered by mere extrapolation. We take care of this artifact with an additional rendering of the occluded area.

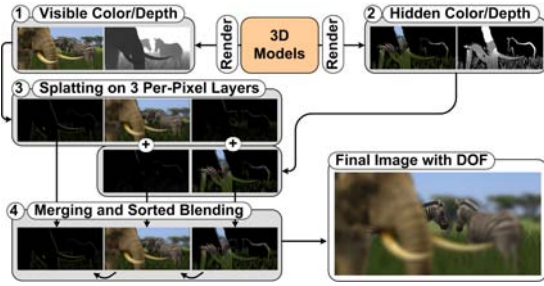


Figure 2: Overview of our DOF rendering framework.

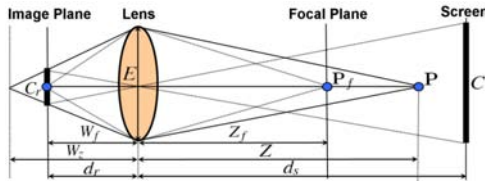


Figure 3: The thin-lens model for depth-of-field [PC81].

Bokeh Rendering “Bokeh” refers to the blurred appearance distorted by the aperture shape and *intensity distribution function* (IDF) of a lens. For bokeh rendering, previous studies proposed a stochastic sampling scheme for the multi-pass rendering [BW02] and a gather method [RTI03]. However, implementing the correct bokeh with the gather is rather difficult (e.g., [KLO06]), since bokeh patterns are determined from the scattered pixels of a source rather than the neighbor pixels to be gathered. In contrast, our method follows the scatter and can simulate the accurate bokeh effects.

3. Depth-of-Field Rendering Algorithm

The procedure of our DOF rendering framework at run-time is illustrated in Figure 2. The circled numbers represent independent steps running on separate GPU programs. First, given a 3D scene, a typical pinhole color/depth image, V , is rendered to a floating-point RGBZ texture using a special GPU program that simultaneously renders color and depth. Second, another color/depth image, H , is rendered for partially occluded areas using another GPU program (Section 3.2). Third, the two pinhole images are separately blurred using the point splatting (Section 3.3). During the splatting, the source pixels of each sprite are written to one of the three layers according to its relative order for the destination pixel depth (Section 3.4). In the last step, the blurred layers are alpha-blended and normalized in the descending order of layer depths (Section 3.5), and then the final image is obtained. The subsequent sections present more details.

3.1. Relating Depth to CoC

We first describe how to compute a CoC diameter from the depth of each pixel, which is frequently used in the subse-

quent computations. Similarly to most post-filtering methods, we use the thin lens model of [PC81] (see Figure 3). The image distances of an object at $\mathbf{P}$ and the focal point ($\mathbf{P}_f$), W_z and W_f , are related to a lens configuration by:

$$W_z = \frac{FZ}{Z-F} \quad \text{and} \quad W_f = \frac{FZ_f}{Z_f-F}, \quad (1)$$

where F is the focal length of the lens. Z and Z_f are the distances from $\mathbf{P}$ and $\mathbf{P}_f$ to the lens, respectively. Then, the CoC diameter on the image plane, C_r , can be computed as:

$$C_r = |W_z - W_f| \frac{E}{W_z} = \left(\frac{EF}{Z_f - F} \right) \frac{|Z - Z_f|}{Z}, \quad (2)$$

where E is the effective lens size. C_r can be further projected onto a pixel-space CoC on the screen, C , as:

$$C = DPI(d_s/d_r)C_r, \quad (3)$$

where DPI represents the number of pixels per unit length. d_s and d_r denote the distances from the lens to the screen and the image plane, respectively. Note that we do not store C to a texture. Instead, the CoC diameter is computed on demand.

3.2. Rendering of Partially Occluded Area

In addition to a normally rendered visible color/depth image (V), we render an additional color/depth image (H) that contains pixels occluded by the visible pixels. This image is greatly useful for producing semitransparent sharp areas behind the blurry foreground. In typical model rendering, hidden surfaces are overwritten due to the depth test in the fixed rendering pipeline. Thus, in order to render the hidden surfaces, a special fragment shader is required, which is similar to the typical fixed-pipeline shader but performs three additional depth tests using V as follows.

Let Z_v be the depth of a pixel in V , and Z' be the depth of a fragment being processed by the new shader. First, a fragment close enough to the visible pixel ($Z' < Z_v + \epsilon$) is rejected. Since we use a single hidden image, a small offset, ϵ , is controlled such that the fragments belonging to the same object are rejected but those belonging to the second nearest object should pass the test for avoiding artifacts. In our implementation, $\epsilon=0.05$ (on the normalized scale) worked well for most cases. Second, if the visible pixel is in the background ($Z_v > Z_f$), the fragment is rejected, because partial occlusion is scarcely perceived in the background. Finally, a completely hidden fragment is rejected, which incurs redundant splatting overhead. If a depth change in the neighborhood of a visible pixel (roughly a CoC; see Figure 4) is very small, the splatted intensity in the pixel is saturated to one. In that case, the occluded fragment becomes completely invisible due to alpha blending (explained later in Section 3.5).

In order to detect the depth change around a pixel position, $\mathbf{p}$, the following operator is applied:

$$\mathcal{L}(\mathbf{p}) = \sum_{\mathbf{n} \in \Omega} (Z_v(\mathbf{n}) - Z_v(\mathbf{p})) / N_\Omega, \quad (4)$$

where Ω is a set of neighbor pixels, and N_Ω is the number

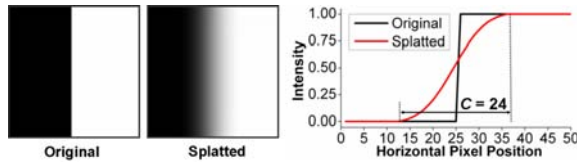


Figure 4: Smooth intensity decrease in the splatted image using CoCs of a 24-pixel diameter for the boundary of the white rectangle. The distance from the edge to the inner area of the full intensity is roughly same to the radius of the CoC.



Figure 5: Accepted pixels after the first and second depth tests (left), and the third test (middle) for the Elephant scene (right). The farthest zebra is focused.

of elements in Ω . This operator is exactly the same as the Laplacian filter (for edge detection), but, instead of typical 4 neighbors, we use random samples within the CoC of $\mathbf{p}$ (e.g., 12 Poisson disk samples [Coo86]). If $\mathcal{L}(\mathbf{p})$ is less than a small threshold (e.g., 0.01 on a normalized scale), the fragment is rejected as completely hidden. Although some fragments accepted in this test may be invisible, they are also removed later during the final blending. Figure 5 shows the example images of pixels accepted with the consecutive tests.

3.3. Image-Space Point Splatting

This section describes how V and H are blurred using the point splatting. The point splatting is a common technique to render a sprite for a 3D point using point sprites present in the GPU, which enlarges a point to a multi-pixel square. The texturing extension of point sprites allows drawing of a circular CoC, even for complex IDF's for bokeh effects.

Given an image, each pixel is mapped to a point in a screen-aligned point array whose dimension is the same as that of the image. When these pixels are enlarged to the size of their CoCs, the overlapped colors stand for the blurred colors.

Due to the scatter of a single pixel on a multi-pixel circle, its intensity should be decreased by its area such that the sum of intensities is equal to one. This intensity decrease is reflected to the alpha channel of the pixel in a sprite (note that the pixel format of a sprite is RGBA unlike RGBZ of V and H). Let $\mathbf{p}$ be the source pixel of a sprite, and $\mathbf{q}$ be the destination position to be written. The alpha value at $\mathbf{q}$, $A(\mathbf{p}, \mathbf{q})$, is proportional to the reciprocal of the CoC area with the diameter, $C(\mathbf{p})$, as:

$$A(\mathbf{p}, \mathbf{q}) = k_n / C(\mathbf{p})^2, \quad \text{where } \|\mathbf{q} - \mathbf{p}\| \leq C(\mathbf{p})/2. \quad (5)$$

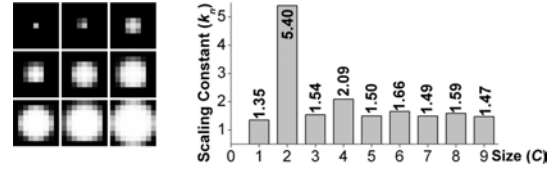


Figure 6: Example rasterization patterns and scaling constants in terms of a point size for the uniform IDF.

k_n is a scaling constant determined by an IDF. For example, a uniform IDF (ideal IDF [Che87]) uses $k_n = 4/\pi$. In order to apply various bokeh patterns, IDF's are stored into textures and read back in the fragment shader.

The problem here is that whereas $C(\mathbf{p})$ is defined in a continuous range, a sprite is rasterized into discrete pixels, relying on specific GPU implementation. The left image in Figure 6 shows examples of such rasterization patterns when the point of intensity one is splatted for each discrete C (from 1 to 9). Thus, instead of directly using $k_n = 4/\pi$, we need to compute $k_n(C)$ for each C such that the sum of pixel intensities is one (see the right graph in the figure). The $k_n(C)$ values are stored in a look-up table (1-D texture) prior to execution, and are read back in the vertex shader at run-time.

Each overlapping sprite is blended using simple accumulative alpha blending. In OpenGL, alpha blending factors are given by GL_SRC_ALPHA, ONE, ONE, ONE for source color, source alpha, destination color, and destination alpha, respectively. Since the single layer splatting blurs a focused boundary and the hidden pixels cannot be represented, we extend the process to the per-pixel layered splatting.

3.4. Splatting on Per-Pixel Layers

A layer decomposition technique considering local depth variation is explained in this section. In general, local depths around a pixel have the following properties: (1) partial occlusion occurs around where adjacent depths of a pixel abruptly change, and (2) the depths are concentrated on a few narrow ranges in most cases. Based on these observations, we group source pixels of similar depths in the incoming sprites to three layers for each destination pixel. When the layers are properly blended (Section 3.5), the partial occlusion and sharply focused boundary are effectively handled. Due to relying on local depth variation rather than global layers [BBGK02, KS07], our method is mostly free of the discretization problem, and the reduced layers can generate the effects similar to those by a number of global layers.

More specifically, an incoming source pixel from $\mathbf{p}$ is assigned to one of the three—in front of ($l=0$), at the same depth ($l=1$), or behind ($l=2$) the visible pixel at $\mathbf{q}$ —layers, as follows.

$$l(\mathbf{p}, \mathbf{q}) = \begin{cases} 0 & B(\mathbf{p}) - B_v(\mathbf{q}) < -\delta C_v(\mathbf{q}) \\ 1 & |B(\mathbf{p}) - B_v(\mathbf{q})| \leq \delta C_v(\mathbf{q}) \\ 2 & B(\mathbf{p}) - B_v(\mathbf{q}) > \delta C_v(\mathbf{q}) \end{cases}, \quad (6)$$



Figure 7: An example of the layer decomposition.

where B is a signed version of C (i.e., $B = K(Z - Z_f)/Z$, $K = \text{DPI}(d_s/d_r)EF/(Z_f - F)$). $B(\mathbf{p})$ is computed from the image currently being processed (V or H), and $B_v(\mathbf{q})$ and $C_v(\mathbf{q})$, from V . δ is a constant for determining whether the incoming pixel from $\mathbf{p}$ belongs to the same depth group as $\mathbf{q}$. These conditions smartly control the decomposition according to the blurring degree at $\mathbf{q}$; that is, if $C_v(\mathbf{q})$ is relatively small, layers are strictly decomposed, but otherwise are loosely decomposed. Regarding the use of δ , δ that is too small assigns the pixels belonging to the same object to different layers, which causes the discretization problem. Conversely, δ that is too large assigns most pixels to a single layer, resulting in no partial occlusion. For most examples in this paper, empirically chosen δ (0.3~0.5) worked well.

According to the index, l , each processed fragment for V is written to one of the three layer images, L^0 , L^1 , or L^2 , and, similarly for those of H , to M^1 or M^2 . For H , M^0 is not defined, because hidden pixels do not occlude any pixels. This separate writing is implemented using the multiple-render-target capability in the GPU. Figure 7 shows example images of the splatted layers. We can observe that the portions of sprites affecting neighbor pixels are well written to the separate layers, even for depth variations in a small area.

Note that we consider only three layers for real-time implementation. Hence, if more than three depth groups are present around a pixel, a hidden area may not be well represented. However, this problem rarely occurs in practice, unless many thin objects overlap together (see Figure 12, for example). If not constrained by a real-time use, this problem can be accurately resolved by additional depth scatter for finding multiple depth layers (possibly, with clustering of the scattered depths).

3.5. Composition of Layers and Normalization

The final step for the DOF rendering is the composition of the blurred layers. The composition follows typical sorted blending from farther to nearer layers [BBGK02, KS07]. Prior to composition, M^1 and M^2 are simply added to L^1 and L^2 , respectively, because they are mostly in similar depths.



Figure 8: Example images blended without (left) and with (right) normalization. Darker and brighter regions in the left image were correctly normalized in the right image.

Then, the images are blended via the following three steps.

$$\begin{aligned} D &\leftarrow L^2 \\ D &\leftarrow L^1 + (1 - L_a^1)D, \\ D &\leftarrow L^0 + (1 - L_a^0)D \end{aligned} \quad (7)$$

where D is the accumulating image, and L_a^0 and L_a^1 are the alpha components of L^0 and L^1 , respectively. Here, D in each step has “hyper” and “hypo” intensities [MvL00] due to the overlapped CoCs of various sizes (see bright/dark regions in Figure 8). Thus, non-zero pixels in each D are divided by their alpha so that each alpha value becomes one. Figure 8 shows a result of the normalization process.

4. Defocused Highlights

Additional feature of our DOF rendering is the simulation of defocused highlights that occur due to the high intensity contrast in real illumination. This effect plays an important role for artistic photorealism, with various bokeh shapes. However, it cannot be well represented in a typical LDR image. In a LDR image, the incoming light intensity exceeding the maximum pixel value (e.g., 1) is truncated to the maximum, and thus the intensity of the resulted CoC becomes smaller than that of the original CoC.

Thus, in order to attain correct defocused highlights, a non-truncated HDR image is required. If so, rendering of the effect is straightforward, similarly to the case of the motion blur with a HDR image [DM97]. Instead of splatting on the truncated image, the splatting is applied on the HDR image, and then the resulting image is tone-mapped to an LDR image. Figure 9 shows example images. Reinhard tone-mapping [RSSF02] ($a=0.72$) was applied before and after splatting for the left and right images, respectively.

Furthermore, although an LDR image lacks accurate intensities, we can mimic the effect using a common trick that extrapolates intensities of bright areas, as described in [Lan02]. For a pinhole LDR image, the color of each pixel, $\mathbf{F}$, is extrapolated by the successive operations as follows.

$$\begin{aligned} \lambda &= 0.3 F_r + 0.59 F_g + 0.11 F_b \\ \lambda_n &= (\lambda - \lambda_0)/(1 - \lambda_0) \\ \mathbf{F} &\leftarrow ((1 - \lambda_n^\beta) + \lambda_n^\beta \gamma) \mathbf{F} \end{aligned} \quad (8)$$

where λ is the luminance of $\mathbf{F}$, computed by red, green, and

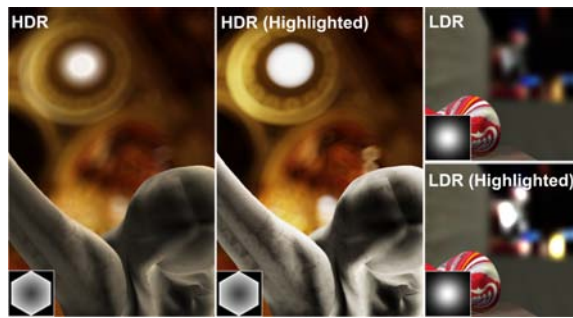


Figure 9: Example results with and without defocused highlights rendered with HDR and LDR images. The bottom-left portion in each image represents the IDF texture used (hexagonal and Gaussian IDFs). Each highlighted image shows better intensity contrast.

blue components: F_r , F_g , and F_b . λ_0 is the threshold for expanding luminance, and λ_n is the renormalized luminance. β is the exponent for falloff, and γ is the expanding gain. This operation is applied only when $\lambda > \lambda_0$. Figure 9 also shows example images generated by this technique. In the figure, the parameters were set as $\lambda_0=0.8$, $\beta=3$, and $\gamma=2$.

5. Acceleration of Rendering Performance

The most pressing obstacle to real-time implementation of our method is the heavy data transfer from the vertex processor to numerous fragments. Unlike the gather approach, the scatter structure cannot exploit a stochastic sampling scheme, because a point should be mapped to the whole sprite. Instead, by reducing the resolution of the point array, rendering performance can be improved.

However, a simple reduction in resolution may result in a magnification artifact around focused regions (e.g., a CoC less than 4 pixels) or where partial occlusion occurs. If we reduce the resolution (e.g., 1/4 of the original) except for such areas, it can be a good alternative of the naive rendering with little perceptual degradation. Since we already have information related to partial occlusion (H) and CoCs, such pixels can be easily found.

In addition, we note that the blurring difference between the two resolutions may cause small holes around overlapping edges. This can be resolved by using one more hidden image for the reduced version, which is similar to H , but the scanning range for $\mathcal{L}$ operator is slightly (e.g., 5%) reduced. This additional hidden image can be simply achieved using H instead of model rendering. Accordingly, the two versions are slightly overlapped, and the artifacts are removed. Figure 10 shows an example. Foreground boundaries and focused regions are blurred at the original resolution, but the rest, at 1/4 reduced resolution.

The remainder of this section reports on the measured rendering performance of our methods with and without the



Figure 10: Rendered images at the original (left) and reduced (right) resolutions. Normalization was not applied.

Table 1: A performance benchmark for the four methods in terms of frame rate for Elephant (E) and Grasshopper (G) scenes. Subscripts represent mean CoC sizes.

Method	E ₈	E ₁₆	E ₃₂	G ₈	G ₁₆	G ₃₂
Accelerated	67	50	31	68	30	17
Not Accelerated	14	4	1	16	5	3
[ZCP07]	105	82	57	144	105	67
[HA90]	12	3	.7	15	4	.9

acceleration, a recent GPU-based method [ZCP07], and the accumulation buffer method [HA90]. We have implemented the four methods using OpenGL and OpenGL Shading Language on a Pentium 2.67GHz Dual-Core with a GeForce 9800GX2. The test was conducted at a 1024×768 resolution, for the Elephant (216,080 triangles) and Grasshopper (46,009 triangles) scenes shown in Figure 1. We used the mean CoC size in an image as a control variable. For the accumulation buffer method, 24, 96, and 384 Poisson disk samples were used for obtaining similar blurring degrees.

Table 1 shows the benchmark result. Our method significantly improved the performance over the non-accelerated method. Zhou's method [ZCP07] showed superior performance as well. The frame rate of the accumulation buffer method rapidly decreased along CoC sizes (i.e., number of model rendering). According to the result, the accelerated version of our method can be used at least up to average 16 pixels of CoC diameters for real-time applications. We note that 16-pixel average of CoCs can represent a larger variation (e.g., up to a CoC diameter of a maximum 24 pixels).

6. Results and Discussion

In this section, we report and discuss the quality of images rendered by our method. Two representatives were already shown in Figure 1. Overall, our method generated natural DOF effects for both foreground and background, without noticeable artifacts. The upper image shows no intensity leakage, similarly to the recent methods [BFSC04, KLO06, ZCP07]. The lower image shows impressive defocused highlights of the hexagonal aperture shape.

With respect to the foreground blurring, we compared our results with Zhou's method (see Figure 11) in terms of the peak signal-to-noise ratio (PSNR). The PSNR was computed as



Figure 11: A comparison of DOF rendering results by our method and those of [ZCP07] and [HA90]. The rounded boxes represent the magnified portions in the images for comparison of foreground blurring.

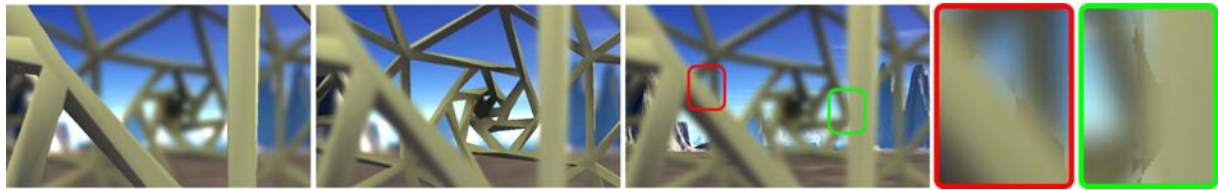


Figure 12: Example images rendered for a scene focused on the near (first), middle, (second), and far (third) objects. Whereas the first two are well represented, the third exhibits small artifacts (see the red and green boxes magnified in the right side).

$10 \log_{10}(255^2/MSE)$. The image rendered by the accumulation buffer method was used as a reference. With our result (using a hidden image), a reasonably high PSNR (36.33 dB) was achieved, whereas Zhou's method and ours (not using a hidden image) showed rather low PSNRs (29.4 and 32.84 dB, respectively). This mostly comes from the difference of foreground representation. At a glance, our result generated with a hidden image is very similar to the reference image, whereas the other two show discontinuous boundaries in the foreground (see the magnified portions). Even with our method, a convincing foreground cannot be achieved without the hidden image (actually, similar to Potmesil's original approach [PC81]).

Recently, several other methods have also reduced blurring discontinuity in the foreground [BTCH05, KLO06, KS07]. However, it seems that they may work only for relatively less blurred scenes that can be covered by extrapolation. Another fundamental problem of these methods is blurring of the focused area to remove sudden blurring changes around the extrapolated boundary, whereas our method successfully maintains the sharply focused area. These problems are similarly applied to the blurring of depth images [BFSC04, EH07], another simpler method against the blurring discontinuity.

In order to assess our results for multiple-layer cases, we provide a case-by-case comparison for a scene where thin geometries overlap together (see Figure 12). Three cases—focused on near, middle, and far objects—were compared. Whereas the first and second cases were well represented, small artifacts (see red boxes) are observed for the third case (the multiple foreground objects of different depths exist around a focused area). This problem results from the limitation of our layer decomposition and hidden image; we



Figure 13: Two example images generated using per-pixel layers (left) and globally defined layers (right) for a scene with long objects.

only use the second nearest hidden image and a single foreground/background layer. As already mentioned, this problem can be resolved using exact depth groups and multiple hidden images, but with longer computational time. Nevertheless, this is not a severe problem in practice, compared to the discontinuity in the foreground boundaries.

As a final example, Figure 13 demonstrates the results when our method is applied to a scene with long objects. Since our method well decomposes even a long object into coherent layers, the discretization artifact is not observed in the figure (see hand rails in the left image). However, such a result might be hard to achieve, if a long object would be allowed to be separated across multiple layers. The right image rendered using globally defined layers [BBGK02] shows an example. This is because the insufficient (less than one) intensities at the layer boundaries can make the colors of a hidden area permeate the boundaries during alpha blending (similarly to the “black band” problem [BTCH05]).

Lastly, we summarize the current limitations of our framework. First, our method requires floating-point texturing capability. However, the difference of acceleration performance for 8-bit and 16-bit textures is practically marginal with an aid of recent GPUs (supporting shader model 4.0). Second, the anti-aliasing is not supported, which is a common problem in most post-filtering methods due to the use of a depth buffer that cannot be anti-aliased. The last limitation is the need of manual tuning of parameters. The performance of our method mainly depends on ϵ (in Section 3.2) and δ (Equation 6). We used $\epsilon=0.05$ and $\delta=0.4$ for most examples in this paper, and tested them for various scenes. Whereas these simple parameters work quite well for moderately blurred scene, a much blurred (e.g., a CoC diameter greater than 32 pixels) scene requires slight tuning of the parameters, according to the spatial configuration of a scene.

7. Conclusion

In this paper, we presented a DOF rendering method using the scatter on per-pixel layers. Our method successfully achieved high-quality DOF effects as well as real-time performance. Our method can be used for post-processing of both real-time and off-line rendered images. For off-line applications, more realistic results would be obtained with an extension to the depth scatter and clustering. Although an existing renderer has to be modified for rendering a hidden image, good real-time performance suggests our framework to be a practical alternative to multi-pass rendering.

Acknowledgments

This work was supported in parts by the KOSEF grant funded by the Korea Government (MOST) (No. R01-2006-000-11142-0 and R0A-2008-000-20087-0), and by the MIC, Korea, under the ITRC support program supervised by the IITA (IITA-2008-C1090-0804-0002). The elephant, zebra, ship, and statue models were taken from [Tas02]. The can and thin geometry models were taken from <http://3d02.com> and <http://sharecg.com/v/15281/3d-model/Ormesh-05>. The grasshopper model and the radiance map were obtained through the courtesy of Toru Miyazawa and Paul Debevec (<http://debevec.org>). Correspondence concerning this article can be addressed to Gerard J. Kim.

References

- [AMMH07] AKENINE-MÖLLER T., MUNKBERG J., HASSELGREN J.: Stochastic rasterization using time-continuous triangles. In *Proc. Graphics Hardware* (2007), pp. 7–16.
- [BBGK02] BARSKY B. A., BARGTEIL A. W., GARCIA D. D., KLEIN S. A.: Introducing vision-realistic rendering. In *Proc. Eurographics Rendering Workshop* (2002), pp. 26–28.
- [BFSC04] BERTALMÍO M., FORT P., SÁNCHEZ-CRESPO D.: Real-time, accurate depth of field using anisotropic diffusion and programmable graphics cards. In *Proc. 3DPVT* (2004), pp. 767–773.
- [BTCH05] BARSKY B., TOBIAS M., CHU D., HORN D.: Elimination of artifacts due to occlusion and discretization problems in image space blurring techniques. *Graphical Models* 67 (2005), 584–599.
- [BW02] BUHLER J., WEXLER D.: A phenomenological model for bokeh rendering. In *ACM SIGGRAPH abstracts and applications* (2002).
- [Che87] CHEN Y. C.: Lens effect on synthetic image generation based on light particle theory. *The Visual Computer* 3, 3 (1987), 125–136.
- [Coo86] COOK R. L.: Stochastic sampling in computer graphics. *ACM Trans. on Graphics* 5, 1 (1986), 51–72.
- [CPC84] COOK R. L., PORTER T., CARPENTER L.: Distributed ray tracing. *Computer Graphics* 18, 3 (1984), 137–145.
- [Dem03] DEMERS J.: Depth of field in the ‘toys’ demo. In *Game Developers Conference* (2003).
- [DM97] DEBEVEC P. E., MALIK J.: Recovering high dynamic range radiance maps from photographs. *Proc. ACM SIGGRAPH* (1997), 369–378.
- [EH07] EARL HAMMON J.: Practical post-process depth of field. In *GPU Gems 3*, Nguyen H., (Ed.). Addison-Wesley, 2007, ch. 28, pp. 583–606.
- [HA90] HAEBERLI P., AKELEY K.: The accumulation buffer: Hardware support for high-quality rendering. In *Proc. ACM SIGGRAPH* (1990), pp. 309–318.
- [KLO06] KASS M., LEFOHN A., OWENS J.: *Interactive Depth of Field Using Simulated Diffusion on a GPU*. Tech. rep., Pixar Animation Studios, 2006.
- [KS07] KRAUS M., STRENGERT M.: Depth-of-field rendering by pyramidal image processing. *Computer Graphics Forum* 26, 3 (2007).
- [Lan02] LANDIS H.: Production-ready global illumination. ACM SIGGRAPH Course Note, 2002.
- [MvL00] MULDER J. D., VAN LIERE R.: Fast perception-based depth of field rendering. In *Proc. VRST* (2000), pp. 129–133.
- [PC81] POTMESIL M., CHAKRAVARTY I.: A lens and aperture camera model for synthetic image generation. *Proc. ACM SIGGRAPH* 15, 3 (1981), 297–305.
- [Rok96] ROKITA P.: Generating depth-of-field effects in virtual reality applications. *IEEE Computer Graphics and its Application* 16, 2 (1996), 18–21.
- [RSSF02] REINHARD E., STARK M., SHIRLEY P., FERWERDA J.: Photographic tone reproduction for digital images. In *Proc. ACM SIGGRAPH* (2002), pp. 267–276.
- [RTI03] RIGUER G., TATARCHUK N., ISIDORO J.: Real-time depth of field simulation. In *ShaderX2: Shader Programming Tips and Tricks with DirectX 9*, Engel W. F., (Ed.). Wordware, 2003, ch. 4, pp. 529–556.
- [Sch04] SCHEUERMANN T.: Advanced depth of field. In *Game Developers Conference* (2004).
- [Sco94] SCOFIELD C.: 2.5d depth-of-field simulation for computer animation. In *Graphics Gems III*, Kirk D., (Ed.). Morgan Kaufman, 1994, ch. 1.8, pp. 36–38.
- [Tas02] *500 3D Objects*. Taschen, 2002. ISBN 4-88783-133-1.
- [ZCP07] ZHOU T., CHEN J. X., PULLEN M.: Accurate depth of field simulation in real time. *Computer Graphics Forum* 26, 1 (2007).